Information Access with Apache Lucene – Part 3

Metodi per il Ritrovamento dell'Informazione

Laurea Triennale in Informatica
Università degli Studi di Bari Aldo Moro

Prof. Cataldo Musto

cataldo.musto@uniba.it

Credits: Marco Polignano

Code Repository & Requirements

Code repository

https://github.com/swapUnibaAcademy/MRI_Lab

Requirements

- Java SDK 1.8+ https://www.java.com/en/download/
- IDE: NetBeans, IntelliJ, Eclipse, …
- **Maven:** https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html

Lucene Search

- **IndexReader:** interface for accessing an index
- **QueryParser:** parses the user query

- **IndexSearcher:** implements search over a single IndexReader
  - `search(Query query, int numDoc)`
  - TopDocs -> result of search
    - `TopDocs.scoreDocs` returns an array of retrieved documents (ScoreDoc)

Lucene Search

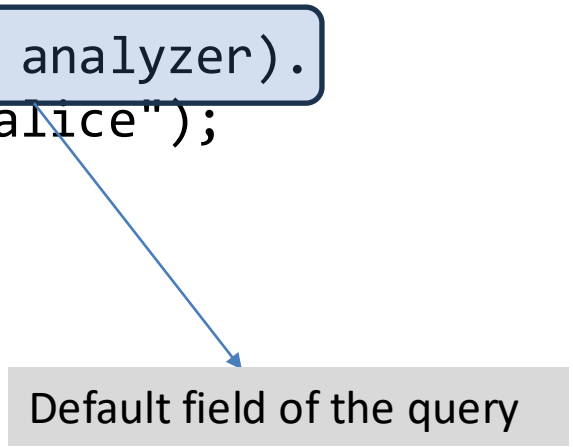
- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer).
parse("chapter:alice OR chapter_text:alice");
```

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer).
parse("chapter:alice OR chapter_text:alice");
```



Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer).
parse("chapter:alice OR chapter_text:alice");
```

We explicitly indicate the elements of the query, by using the syntax **Field:value**

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer)
        .parse("alice");
```

- **MultiFieldQueryParser**

```
QueryParser parser = new MultiFieldQueryParser
(new String[] {"author", "chapter", "chapter_text"}, analyzer);

query = parser.parse("alice");
```

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer)
        .parse("alice");
```

Name = Alice

- **MultiFieldQueryParser**

```
QueryParser parser = new MultiFieldQueryParser
(new String[] {"author", "chapter", "chapter_text"}, analyzer);

query = parser.parse("alice");
```

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = new QueryParser("name", analyzer).
      parse("alice OR chapter_text:alice");
```

Name = Alice OR chapter_text = Alice

- **MultiFieldQueryParser**

```
QueryParser parser = new MultiFieldQueryParser
(new String[] {"author", "chapter", "chapter_text"}, analyzer);

query = parser.parse("alice");
```

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = newQueryParser("name", analyzer).
      parse("chapter:alice OR chapter_text:alice");
```

- **MultiFieldQueryParser**

Ignores the default field

```
QueryParser parser = new MultiFieldQueryParser
(new String[] {"author", "chapter", "chapter_text"}, analyzer);

query = parser.parse("alice");
```

Lucene Search

- We have several strategies to formulate the query
- **QueryParser**

```
Query query = newQueryParser("name", analyzer).
       parse("chapter:alice OR chapter_text:alice");
```

- **MultiFieldQueryParser**

```
QueryParser parser = new MultiFieldQueryParser
(new String[] {"author", "chapter", "chapter_text"}, analyzer);

query = parser.parse("alice");
```

We indicate the fields as a vector

Lucene Search

- We have several strategies to formulate the query
- **BooleanQuery**

```
BooleanQuery.Builder qb = new BooleanQuery.Builder();

qb.add(new TermQuery(new Term("chapter", "alice")),
BooleanClause.Occur.SHOULD);

qb.add(new TermQuery(new Term("chapter_text", "alice")),
BooleanClause.Occur.SHOULD);

Query q = qb.build();
```

Lucene Search

- We have several strategies to formulate the query
- **BooleanQuery**

```
BooleanQuery.Builder qb = new BooleanQuery.Builder();

qb.add(new TermQuery(new Term("chapter", "alice")),
BooleanClause.Occur.SHOULD);

qb.add(new TermQuery(new Term("chapter_text", "alice")),
BooleanClause.Occur.SHOULD);

Query q = qb.build();
```

We add several TermQuery objects to the query builder

Lucene Search

- We have several strategies to formulate the query
- **BooleanQuery**

```
BooleanQuery.Builder qb = new BooleanQuery.Builder();

qb.add(new TermQuery(new Term("chapter", "alice")),
BooleanClause.Occur.SHOULD);

qb.add(new TermQuery(new Term("chapter_text", "alice")),
BooleanClause.Occur.SHOULD);

Query q = qb.build();
```

SHOULD --> OR
MUST --> AND

Lucene Search

See the class `uniba.mri.lucene.TestSearch1`

```
//Open Index and create a Searcher
FSDirectory fsdir = FSDirectory.open(new File("./resources/alice").toPath());
IndexSearcher searcher = new IndexSearcher(DirectoryReader.open(fsdir));

//Single term query
//Query q = new TermQuery(new Term("chapter", "rabbit"));

//Boolean query
BooleanQuery.Builder qb = new BooleanQuery.Builder();
qb.add(new TermQuery(new Term("chapter", "rabbit")),
                                    BooleanClause.Occur.SHOULD);
qb.add(new TermQuery(new Term("chapter_text", "alice")),
                                    BooleanClause.Occur.SHOULD);
Query q = qb.build();

//Search Results
TopDocs topdocs = searcher.search(q, 10);
System.out.println("Found " + topdocs.totalHits + " document(s).");
```

Lucene Search

See the class `uniba.mri.lucene.TestSearch1`

```
//Open Index and create a Searcher
FSDirectory fsdir = FSDirectory.open(new File("./resources/alice").toPath());
IndexSearcher searcher = new IndexSearcher(DirectoryReader.open(fsdir));

//Single term query
//Query q = new TermQuery(new Term("chapter", "rabbit"));

//Boolean query
BooleanQuery.Builder qb = new BooleanQuery.Builder();
qb.add(new TermQuery(new Term("chapter", "rabbit")),
                                    BooleanClause.Occur.SHOULD);
qb.add(new TermQuery(new Term("chapter_text", "alice")),
                                    BooleanClause.Occur.SHOULD);
Query q1 = qb.build();

//Search Results
TopDocs topdocs = searcher.search(q, 10);
System.out.println("Found " + topdocs.totalHits.value + " document(s).");
```

We open the index we previously created

More details on query syntax

- **Wildcard Searches**
  - tes* (test - tests - tester)
  - te?t (test – text)
  - te*t (tempt)
  - tes?
- **Fuzzy Searches (Levenshtein Distance) (TERM)**
  - roam~ (foam – roams)
  - roam~0.8
- **Range Searches**
  - mod_date:[20020101 TO 20030101]
  - title:{Aida TO Carmen}
- **Proximity Searches (PHRASE)**
  - "jakarta apache"~10

More details on query syntax

- **Boosting a Term**
  - jakarta^4 apache
  - "jakarta apache"^4 "Apache Lucene”
- **Boolean Operator**
  - NOT, OR, AND
  - + required operator
    title:(+return +"pink panther")
  - - prohibit operator
  - Escaping Special Characters by \

DELETING DOCUMENTS

IndexWriter

- `deleteDocuments(Term… terms)`
- `deleteDocuments(Query… queries)`
- `updateDocument(Term term, Iterable<? extends IndexableField> doc)`
  Updates a document by first deleting the document(s) containing term and then adding the new document.
- `updateDocuments(Term delTerm, Iterable<? extends Iterable<? extends IndexableField>> docs)`
  Deletes and adds a block of documents with sequentially assigned document IDs, such that an external reader will see all or none of the documents.
- deleting does not immediately reclaim space

Search Engine - Home Work

- Create your own search engine
- Collect (or crawl) enough text documents containing information you want to index (i.e., sports news)
- **Optional:** define the fields and index them separately
- Index the content and make some queries

Develop a “little” search engine

FIRST PROJECT

Exercise – Search Engine

Build a “little” search engine that indexes and searches text files into a folder

- main class `IndexSE` takes the folder name and the index directory name as arguments
  - the class indexes all the “.txt” into the folder
- main class `SearchSE` takes the index directory name and the query as arguments
  - the class searches the index

Exercise – Search Engine

See classes:

- `di.uniba.mri.lucene.se.IndexSE`
- `di.uniba.mri.lucene.se.SearchSE`

IndexSE

```
FSDirectory fsdir = FSDirectory.open(new File(./index_path).toPath());
//creiamo l'indice
IndexWriterConfig iwc = new IndexWriterConfig(new StandardAnalyzer());
iwc.setOpenMode(IndexWriterConfig.OpenMode.CREATE);
IndexWriter writer = new IndexWriter(fsdir, iwc);
File dir = new File(./path_to_files)
File[] files = dir.listFiles();

for (File file : files) {
   if (file.isFile() && file.getName().endsWith(".text"))
       doc.add(new StringField("id", file.getAbsolutePath(),
                   Field.Store.YES));
       doc.add(new TextField("text", new FileReader(file)));
       writer.addDocument(doc);
    }
}
writer.close();
```

IndexSE

Passiamo come parametro la directory

```
FSDirectory fsdir = FSDirectory.open(new File(./index_path).toPath());
//creiamo l'indice
IndexWriterConfig iwc = new IndexWriterConfig(new StandardAnalyzer());
iwc.setOpenMode(IndexWriterConfig.OpenMode.CREATE);
IndexWriter writer = new IndexWriter(fsdir, iwc);
File dir = new File(./path_to_files)
File[] files = dir.listFiles();

for (File file : files) {
   if (file.isFile() && file.getName().endsWith(".text"))
       doc.add(new StringField("id", file.getAbsolutePath(),
                   Field.Store.YES));
       doc.add(new TextField("text", new FileReader(file)));
       writer.addDocument(doc);
    }
}
writer.close();
```

IndexSE

```
FSDirectory fsdir = FSDirectory.open(new File(args[1]).toPath());
//creiamo l'indice
IndexWriterConfig iwc = new IndexWriterConfig(new StandardAnalyzer());
iwc.setOpenMode(IndexWriterConfig.OpenMode.CREATE);
IndexWriter writer = new IndexWriter(fsdir, iwc);
File dir = new File(./path_to_files)
File[] files = dir.listFiles();

for (File file : files) {
   if (file.isFile() && file.getName().endsWith(".text"))
       doc.add(new StringField("id", file.getAbsolutePath(),
                   Field.Store.YES));
       doc.add(new TextField("text", new FileReader(file)));
       writer.addDocument(doc);
    }
}
writer.close();
```

Prendiamo tutti i .txt della directory

IndexSE

```
FSDirectory fsdir = FSDirectory.open(new File(args[1]).toPath());
//creiamo l'indice
IndexWriterConfig iwc = new IndexWriterConfig(new StandardAnalyzer());
iwc.setOpenMode(IndexWriterConfig.OpenMode.CREATE);
IndexWriter writer = new IndexWriter(fsdir, iwc);
File[] files = dir.listFiles();

for (File file : files) {
   if (file.isFile() && file.getName().endsWith(".txt")) {
       Document doc = new Document();
       doc.add(new StringField("id", file.getAbsolutePath(),
                   Field.Store.YES));
       doc.add(new TextField("text", new FileReader(file)));
       writer.addDocument(doc);
    }
}
writer.close();
```

Mettiamo **tutto il contenuto del file** di testo in un campo.

SearchSE

```
FSDirectory fsdir = FSDirectory.open(new File(args[0]).toPath());

IndexSearcher searcher = new IndexSearcher(DirectoryReader.open(fsdir));
QueryParser qp = new QueryParser("text", new StandardAnalyzer());

//Parse the query
Query q = qp.parse(args[1]);
//Search
TopDocs topdocs = searcher.search(q, 10);
 for (ScoreDoc sdoc : topdocs.scoreDocs)
{
   searcher.doc(sdoc.doc).get("id")
   System.out.println("Found doc, path=" + ", score" + sdoc.score);

}
```

SearchSE

Primo Parametro, directory dell'indice

```
FSDirectory fsdir = FSDirectory.open(new File(args[0]).toPath());

IndexSearcher searcher = new IndexSearcher(DirectoryReader.open(fsdir));
QueryParser qp = new QueryParser("text", new StandardAnalyzer());

//Parse the query
Query q = qp.parse(args[1]);
//Search
TopDocs topdocs = searcher.search(q, 10);
 for (ScoreDoc sdoc : topdocs.scoreDocs)
{
   searcher.doc(sdoc.doc).get("id")
    System.out.println("Found doc, path=" + ", score" + sdoc.score);

}
```

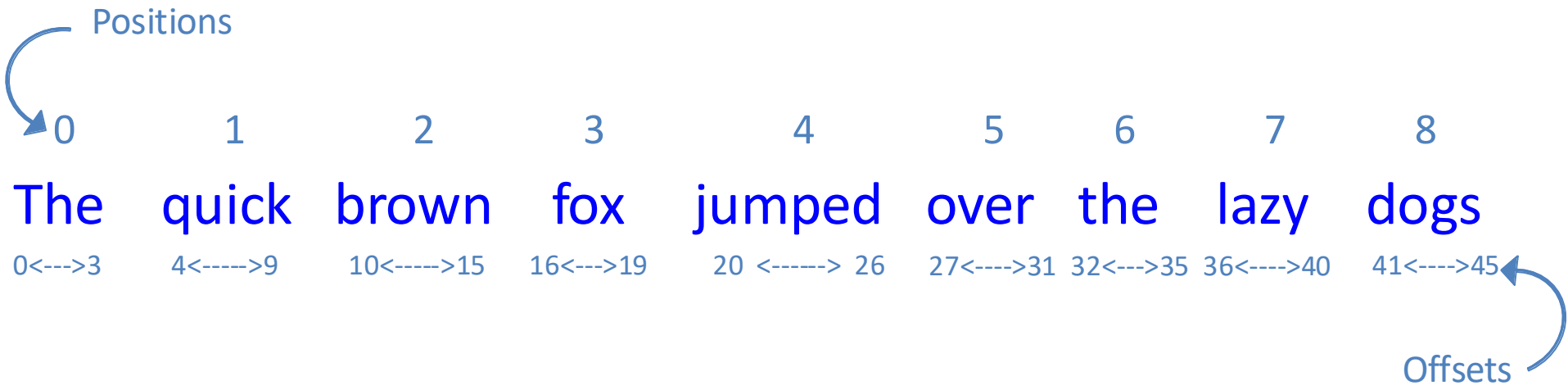
Secondo parametro, la query

POSTING API

Posting API

- Getting access to term **frequency, positions and offsets**
- Required for:
  - Relevance Feedback and “More Like This”
  - Clustering
  - Similarity between two documents
  - Highlighter
    - needs offsets info

Basics: Positions and Offsets



Posting API

- `Fields` is the initial entry point into the postings APIs

- `Terms` represents the collection of terms within a field
- `TermsEnum` provides an iterator over the list of terms within a field

- `PostingsEnum` is an extension of `DocIdSetIterator` that iterates over the list of documents for a term, along with the term frequency within that document

- `PostingsEnum` also allows iteration of the positions a term occurred within the document, and any **additional per-position information (offsets and payload)**

Store Posting 1/2

- During indexing, create a `Field` that stores TermVectors through `FieldTyp` (we used TextField as a predefined IndexableField)
  - `FieldType ft = new FieldType(TextField.TYPE_NOT_STORED);`
    - `TYPE_STORED|TYPE_NOT_STORED`
- Invoke in cascade the following methods to set the field type:
  - `ft.setTokenized(true); //done as default`
  - `ft.setStoreTermVectors(true);`
  - `ft.setStoreTermVectorPositions(true);`
  - `ft.setStoreTermVectorOffsets(true);`
  - `ft.setStoreTermVectorPayloads(true);`

Store Posting 2/2

See class: `di.uniba.it.mri2324.lucene.se.post.IndexPostExample`

```
//define a custom field type that stores post information
FieldType ft = new FieldType(TextField.TYPE_NOT_STORED);
ft.setStoreTermVectors(true);
ft.setStoreTermVectorPositions(true);
ft.setStoreTermVectorOffsets(true);
…
Document doc1 = new Document();
doc1.add(new StringField("id", "1", Field.Store.YES));
doc1.add(new Field("text", new FileReader("./resources/text/es1.txt"), ft));
writer.addDocument(doc1);
```

Store Posting 2/2

See class: `di.uniba.it.mri2324.lucene.se.post.IndexPostExample`

```
//define a custom field type that stores post information
FieldType ft = new FieldType(TextField.TYPE_NOT_STORED);
ft.setStoreTermVectors(true);
ft.setStoreTermVectorPositions(true);
ft.setStoreTermVectorOffsets(true);
…
Document doc1 = new Document();
doc1.add(new StringField("id", "1", Field.Store.YES));
doc1.add(new Field("text", new FileReader("./resources/text/es1.txt"), ft));
writer.addDocument(doc1);
```

Il nuovo campo viene memorizzato **utilizzando il formato appena definito.**

Posting API

- Getting the BoW of a document: di.uniba.it.mri2324.lucene.se.post.PostExample

```
Fields fields = ireader.getTermVectors(docid);

for (String field : fields) {
      Terms terms = fields.terms(field);
      TermsEnum termsEnum = terms.iterator();
       BytesRef term = null;
       while ((term = termsEnum.next()) != null)
            {
            System.out.print(term.utf8ToString());
            PostingsEnum postings = termsEnum.postings(null,PostingsEnum.FREQS);

            while (postings.nextDoc() != DocIdSetIterator.NO_MORE_DOCS) {
                System.out.println(":" + postings.freq());
            }
       }
}
```

More info
https://lucene.apache.org/core/8_6_2/core/org/apache/lucene/index/package-summary.html#postings

Posting API

- Getting the BoW of a document:
  di.uniba.it.mri2324.lucene.se.post.PostExample

```
Fields fields = ireader.getTermVectors(docid);

for (String field : fields) {
      Terms terms = fields.terms(field);
      TermsEnum termsEnum = terms.iterator();
      BytesRef term = null;
      while ((term = termsEnum.next()) != null)
           {
           System.out.print(term.utf8ToString());
           PostingsEnum postings = termsEnum.postings(null,PostingsEnum.FREQS);

           while (postings.nextDoc() != DocIdSetIterator.NO_MORE_DOCS) {
               System.out.println(":" + postings.freq());
           }
      }
}
```

ID del documento

More info
https://lucene.apache.org/core/8_6_2/core/org/apache/lucene/index/package-summary.html#postings

Posting API

- Getting the BoW of a document: `di.uniba.it.mri2324.lucene.se.post.PostExample`

```
Fields fields = ireader.getTermVectors(docid);

for (String field : fields) {
      Terms terms = fields.terms(field);
      TermsEnum termsEnum = terms.iterator();
      BytesRef term = null;
      while ((term = termsEnum.next()) != null)
           {
           System.out.print(term.utf8ToString());
           PostingsEnum postings = termsEnum.postings(null,PostingsEnum.FREQS);

           while (postings.nextDoc() != DocIdSetIterator.NO_MORE_DOCS) {
               System.out.println(":" + postings.freq());
           }
      }
}
```

Accediamo alle frequenze dei termini dentro `termsEnum`

More info
https://lucene.apache.org/core/8_6_2/core/org/apache/lucene/index/package-summary.html#postings

Posting API

- Getting the BoW of a document: `di.uniba.it.mri2324.lucene.se.post.PostExample`

```
Fields fields = ireader.getTermVectors(docid);

for (String field : fields) {
      Terms terms = fields.terms(field);
      TermsEnum termsEnum = terms.iterator();
      BytesRef term = null;
      while ((term = termsEnum.next()) != null)
           {
           System.out.print(term.utf8ToString());
           PostingsEnum postings = termsEnum.postings(null,PostingsEnum.ALL);

           while (postings.nextDoc() != DocIdSetIterator.NO_MORE_DOCS) {
               System.out.println(":" + postings.freq());

               for (int i = 0; i < postings.freq(); i++) {
                            int position = postings.nextPosition();
                            System.out.println("\t" + position + "\t{" +
                           postings.startOffset() + ", " + postings.endOffset()
               + "}");}}}}
```

Accediamo a tutte le informazioni sui termini (es. la posizione) dentro `termsEnum`

Posting APIs - Home Work

- **Integrate** Posting APIs with the search engine previously created

- **Adapt the indexing process**, by using Posting APIs to **store offsets and positions** (create a new field, for example for the text of the chapter in Alice in Wonderland)

- **Pick the top-1 result for your query**, and look for the terms of the query in the text, and return **the position of the terms**.

Thank you!

